1 Decoding the recovered MIPS programs

Figure 1 shows the programs recovered by MIPS-evolved PySR on Parity-last 4, Alternating-last 4, and Base-5 addition. Throughout, $h = (h_0, h_1, \ldots)$ is the RNN hidden state after projection onto its learned integer lattice, primes denote the next state, x (or the digit pair a, b) is the current input, and y is the output read from the new state. To interpret each program we labeled every lattice state with the input histories that reach it, and then checked each recovered equation case by case against that labeling.

Parity-last 4. The 16 states correspond to the 16 windows $d\,a\,b\,c$ of the last four bits (oldest to newest). The coordinate h_2 equals $4 \cdot \text{parity}(d, a, b, c)$; the factor of four is only the lattice refinement scale used for this task (the unrefined lattice merges states), so $h_2 \in \{0, 4\}$ and the output $y = \min(1, h_2)$ simply reads out the parity bit. The pair (h_0, h_1) is a shift register over the last three bits $a\,b\,c$:

abc	000	100	010	110	001	101	011	111
(h_0, h_1)	$(2\|3, 0)$	$(0, 1)$	$(0, 2)$	$(4, 2)$	$(0, 5)$	$(3\|4, 5)$	$(8, 7)$	$(4, 9)$

where $2|3$ and $3|4$ mark small RNN-induced variation that depends on the bit just dropped. The parity update is the algorithmic core. The next window is $a\,b\,c\,x$, so the new parity is $\text{parity}(a, b, c) \oplus x$ and does not depend on the old h_2. On this embedding the three-bit parity is linearly separable: $h_1 < 2h_0$ holds exactly when $\text{parity}(a, b, c) = 0$. The recovered $h_2' = 4\,\text{eq}(\text{lt}(h_1, 2h_0), x)$ therefore computes XNOR of this test with x, i.e. $4\,(\text{parity}(a, b, c) \oplus x)$. The coordinate h_1 essentially encodes the last two bits (b, c) ($\{0, 1\} \to 00$, $2 \to 10$, $5 \to 01$, $\{7, 9\} \to 11$), and h_1' shifts this code and pushes in x: for $x = 0$ it reduces to $\lfloor \min(0.66h_1, 2.32) \rfloor$, and for $x = 1$ to $\lfloor \min(2.32 + \max(3, h_1), 9.32) \rfloor$ (the $2.98567x$ term is inactive because the inner minimum saturates). The values of h_0 follow no simple bit pattern, and the recovered h_0' acts as a lookup table: its cubic branch in h_1 is small only for $h_1 \in \{0, 1\}$, and it also handles the single case in which h_0' depends on h_0 through $\max(h_1, h_0)$, while the second branch supplies the values $\{0, 4, 8\}$ for $h_1 \geq 2$, with the constant 0.227 chosen so that the rounded products have the required parity. This is the only subproblem that neither baseline nor SRBench-evolved PySR recovers.

Alternating-last 4. Here $y = 1$ iff the last four bits alternate (0101 or 1010). The RNN stores the last bit and the length L of the current alternating run in h_0, and a flag $h_1 = \mathbf{1}[L \geq 4]$, so $y = h_1$:

h_0	$6, 7, 8$	1	0	$31, 32$	$37\,(h_1{=}0)$	38	$37\,(h_1{=}1)$
last bit	1	1	1	0	0	0	0
L	1	2	≥ 3	1	2	3	≥ 4

Small values of h_0 encode a last bit of 1 and large values (31–38) a last bit of 0; the states $\{6, 7, 8\}$ and $\{31, 32\}$ are redundant copies with identical transitions. Up to a ± 1 correction, the recovered h_0' is the affine map $h_0' \approx \lfloor 7.83 - h_0/5 + 30 \cdot \mathbf{1}[x{=}0] \rfloor$, which moves the state between the two bands. For $x = 1$, a large h_0 (the run continues) maps to 1 ($L = 2$) or 0 ($L \geq 3$), and a small h_0 (a repeated bit, which resets the run) maps to $\{6, 7, 8\}$. For $x = 0$, the offset of 30 lands in the large band: a repeated 0 maps to $\{31, 32\}$, and a run ending in 1 maps to 37 or 38. The term $\text{eq}(\text{not}(h_0), x)$ corrects the case $h_0 = 0$, $x = 0$, which reaches $L \geq 4$. The recovered h_1' sets the flag exactly when the run already had length at least three and x continues it: the factor $\text{zero}(\text{xor}(\text{lt}(x, h_0), x))$ is one iff ($x = 0$ and $h_0 = 0$) or ($x = 1$ and $h_0 \geq 2$), and the second factor keeps only the $L \geq 3$ states ($h_0 = 0$, $h_0 = 38$, or $h_0 = 37$ with $h_1 = 1$). The old flag h_1 is needed because $h_0 = 37$ alone does not distinguish $L = 2$ from $L \geq 4$.

Base-5 addition. Each step reads one digit of each summand, $a, b \in \{0, \ldots, 4\}$, least significant first, and outputs the corresponding digit of the sum. The ten states correspond to the ten values of the column sum $s = a + b + c_{\text{in}} \in \{0, \ldots, 9\}$, encoded as a carry bit $h_1 = \mathbf{1}[s \geq 5]$ and a digit coordinate $h_0 = 6 - s + 3h_1$ that decreases in s and jumps at the carry ($s = 0, \ldots, 4 \mapsto 6, \ldots, 2$; $s = 5, \ldots, 9 \mapsto 4, \ldots, 0$). The output $y = |h_0 + h_1 - 6| - h_1$ inverts this encoding: without a carry it gives $|-s| = s$, and with a carry $|4 - s| - 1 = s - 5$, i.e. $y = s \bmod 5$. The carry update $h_1' = \mathbf{1}[a + b + h_1 \geq 5]$ is implemented by two branches. Without an incoming carry the mod term vanishes and $\text{eq}(1, 0.119(a+b))$ fires iff $a + b \geq 5$. With an incoming carry, the term $\text{mod}(-1.033, 0.254(a+b))$ equals ≈ 1.0 for $a + b = 4$ and 0.49, just below the 0.5 threshold of eq, for $a + b \leq 3$; this is correct on all inputs but relies on a numerical coincidence. The digit update computes $6 - s' + 3 \cdot \mathbf{1}[s' \geq 5]$ for $s' = a + b + h_1$ as the maximum of a linear branch $6.06 - s'$, which is correct for $s' < 5$, and a mod branch: for $s' \geq 5$, two periods of 4.38 give $\text{mod}(0.24 - s', 4.38) \approx 9 - s'$, the wrapped code. The same pattern, a linear branch plus a mod branch that implements the "subtract the base" wrap-around, appears in the recovered digit updates for base-4 and base-7 addition.

Summary. In all three tasks the RNN representation is a genuine algorithm: a shift register with a running parity, the last bit with an alternating run length, and the column sum stored as a carry and a reversed digit. The outputs, the parity rule $h_1 < 2h_0$, and the carry thresholds are compact and interpretable. The remaining coordinates are recovered as correct piecewise lookups built from floor, mod, and max, which often exploit numerical coincidences or the RNN's arbitrary and partly redundant lattice coordinates; these are precisely the subproblems that baseline and SRBench-evolved PySR tend to miss.

	Base	Evolved	MIPS	Discovered equation (MIPS-evolved)
Parity-last 4. $y =$ parity of the last 4 input bits x. $h_0 \in \{0,2,3,4,8\}$, $h_1 \in \{0,1,2,5,7,9\}$, $h_2 \in \{0,4\}$ (16 states); $x, y \in \{0,1\}$. State: (h_0, h_1) holds the last 3 bits; $h_2 = 4 \cdot$ parity of the last 4.				
h_0'	0/10	0/10	**1/10**	$\min\big(((\max(h_1,h_0)-x)h_1-x)h_1-x+2,\ 4\,\lvert\mathrm{xor}(h_1(0.227249-x),h_1)+x-\mathrm{zero}(h_1)\rvert\big)$
h_1'	**5/10**	**3/10**	**10/10**	$\lfloor\min\big(\min(2.98567x+0.660363h_1, 2.32073)+x\max(3.00001,h_1),\ 9.32075\big)\rfloor$
h_2'	**10/10**	**10/10**	**10/10**	$4\,\mathrm{eq}(\mathrm{lt}(h_1,2h_0),x)$
y	**10/10**	**10/10**	**10/10**	$\min(1,h_2)$
	0/10	0/10	**1/10**	*seeds recovering all subproblems together*
Alternating-last 4. $y = 1$ iff the last 4 input bits x alternate. $h_0 \in \{0,1,6,7,8,31,32,37,38\}$, $h_1 \in \{0,1\}$ (11 states); $x, y \in \{0,1\}$. State: h_0 holds the last bit and the alternating run length L; $h_1 = \mathbf{1}[L \geq 4]$.				
h_0'	**1/10**	0/10	**5/10**	$\lfloor\mathrm{eq}(\mathrm{not}(h_0),x)+30\,\mathrm{not}(x)-0.203372\,h_0+7.82772\rfloor$
h_1'	**1/10**	**10/10**	**10/10**	$\mathrm{zero}(\mathrm{xor}(\mathrm{lt}(x,h_0),x))\cdot\max(h_0-37,\ \max(\mathrm{zero}(x),h_1))$
y	**10/10**	**10/10**	**10/10**	h_1
	0/10	0/10	**5/10**	*seeds recovering all subproblems together*
Base-5 addition. Inputs a, b are digits of two numbers (least significant first); y is the sum digit. $h_0 \in \{0,\ldots,6\}$, $h_1 \in \{0,1\}$ (10 states); $a, b, y \in \{0,\ldots,4\}$. State: column sum $s = a + b + c_{\mathrm{in}}$, stored as carry $h_1 = \mathbf{1}[s \geq 5]$ and digit $h_0 = 6 - s + 3h_1$.				
h_0'	0/10	**2/10**	**3/10**	$\lfloor\max(6.06243-(a+b+h_1),\ \mathrm{mod}(0.243552-(a+b+h_1),\ 4.38026))\rfloor$
h_1'	**8/10**	**10/10**	**10/10**	$\mathrm{eq}(1,\ \max(0.119279(a+b),\ \mathrm{mod}(-1.03325,\ 0.25413(a+b)h_1)))$
y	**10/10**	**10/10**	**9/10**	$\lvert h_0+h_1-6\rvert-h_1$
	0/10	**2/10**	**3/10**	*seeds recovering all subproblems together*

Figure 1: Programs recovered from RNNs by MIPS-evolved PySR on three MIPS tasks. Each task is split into scalar regression problems, one per next-state coordinate (h_i') and one for the output (y). Counts give exact recovery out of ten search seeds for baseline PySR (Base), SRBench-evolved PySR (Evolved), and MIPS-evolved PySR (MIPS). Equations are regrouped, simplified using the observed value sets, and rounded to six significant digits; each displayed form is verified on the complete recorded relation. Operators: $\mathrm{lt}(u,v) = \mathbf{1}[u < v]$, $\mathrm{eq}(u,v) = \mathbf{1}[\lvert u-v\rvert < 0.5]$, $\mathrm{zero}(u) = \mathrm{not}(u) = \mathbf{1}[\lvert u\rvert < 0.5]$, $\mathrm{mod}(u,v) = u \bmod \lvert v\rvert$, $\mathrm{xor}(u,v) = (\mathrm{round}(u) + \mathrm{round}(v)) \bmod 2$. See Appendix 1 for a full decoding of each program.